

Problem 1 - Washing Machine Mount Design

The Matlab code for this problem is listed in Appendix 1.

Problem 1a

Equation calculates the force produced by the imbalanced 1kg mass at a 0.5 m radius from the center of the washing machine as a function of angular velocity,

$$F = m \cdot \omega^2 \cdot r \quad (1)$$

where m is 0.5 kg, ω is the angular velocity in $\frac{\text{radians}}{\text{s}}$, and r is 0.5 m.

Figure 1 shows the force produced by the 1 kg load as a function of angular velocity in RPM.

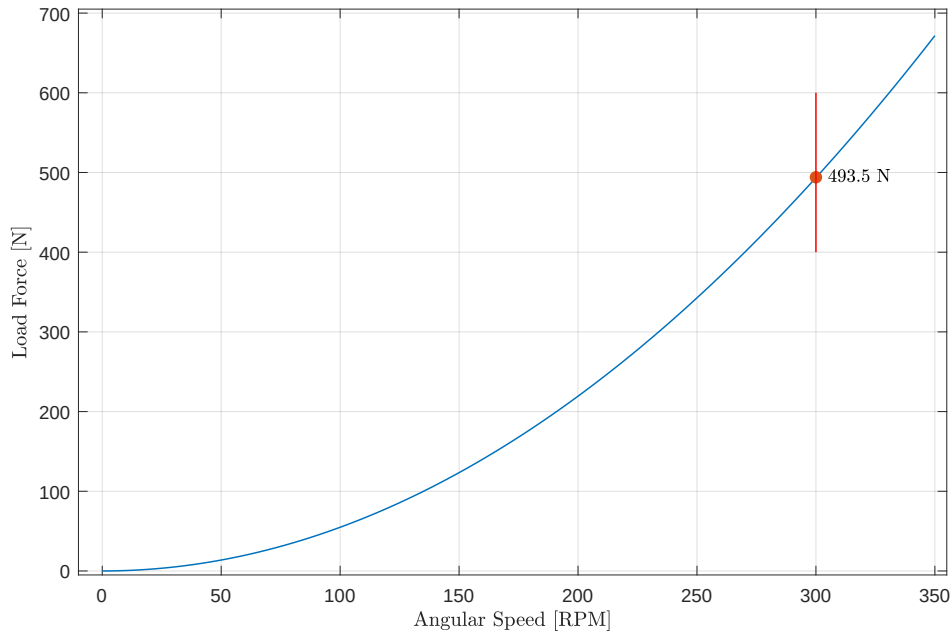


Figure 1: Load force versus washing machine angular speed.

At 300 RPM, or 5 Hz, the imbalanced load force amplitude is 493.5 N.

Problem 1b

The required spring stiffness, k_s , is $9,810 \frac{\text{N}}{\text{m}}$. The mount's natural frequency, w_o , is 1.5 Hz.

Problem 1c

Figure 2 shows the transmission loss of the system for a set of damping ratios ranging from 0.001 to 1.

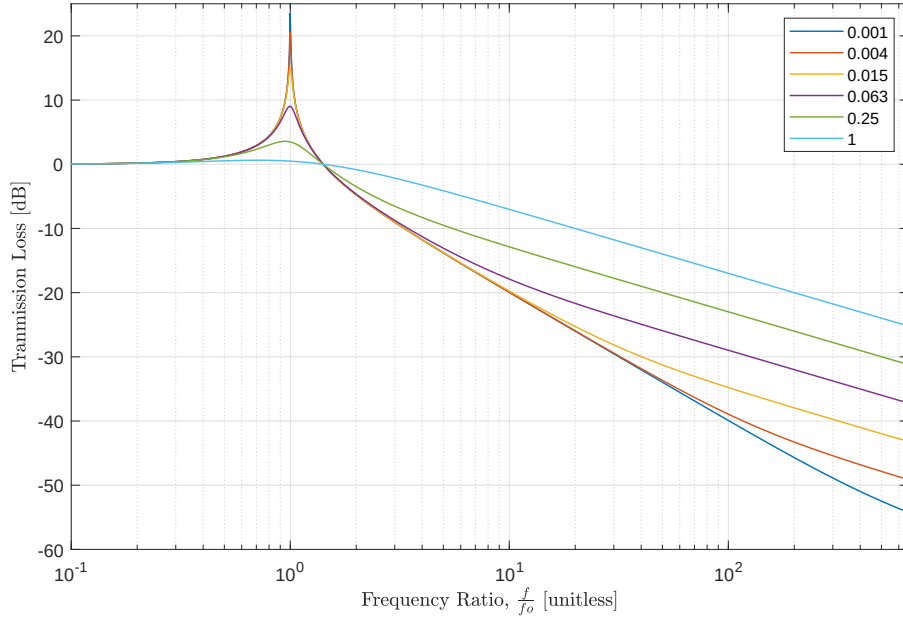


Figure 2: Transmission loss as a function of damping ratio. The natural frequency is 1.5 Hz.

Problem 1d

Figure 3 shows the force applied to the foundation.

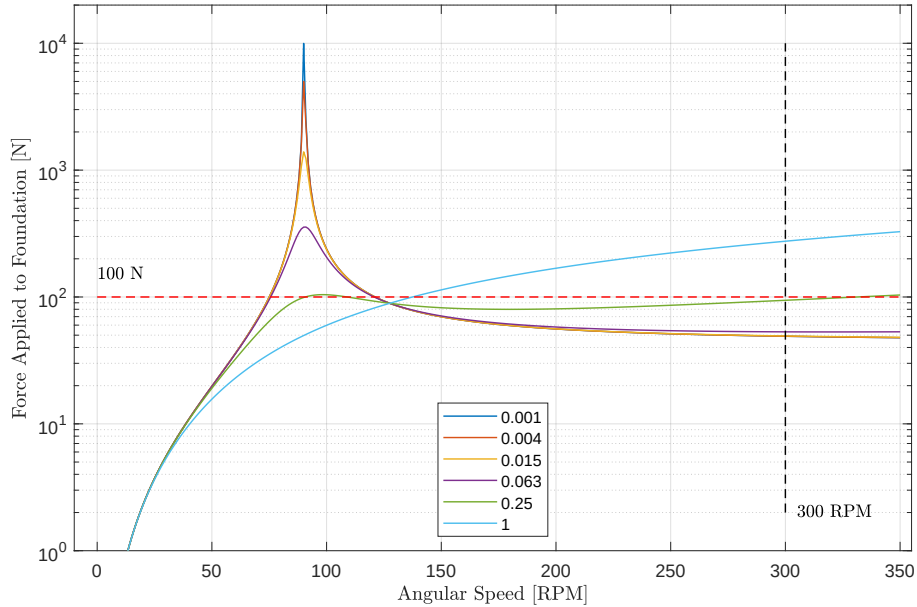


Figure 3: Force applied to the foundation as a function of damping ratio.

No damping ratio makes the applied force less than 100 N across the 0 to 350 RPM operating range.

The best damping ratio is $\zeta = 0.25$. The force applied to the foundation still exceeds the 100 N target in only two small RPM regions: 91 to 109, and 331 to 350 RPM. In both of these regions the force is approximately 104 N.

Problem 1e

The viscous damping coefficient is $519.4 \frac{\text{kg}}{\text{s}}$ based on a total mass of 110 kg.

Figure 4 shows the admittance.

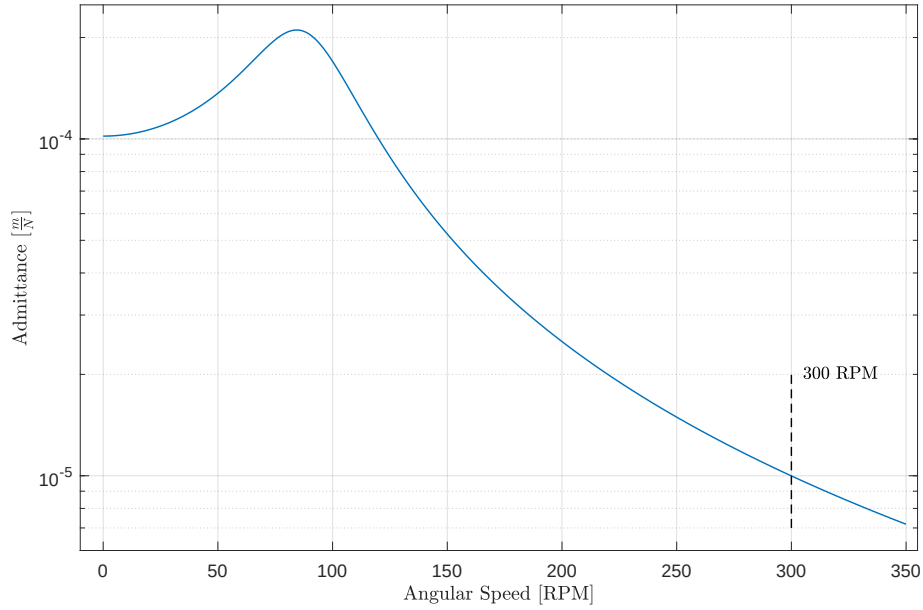


Figure 4: Admittance versus RPM.

The admittance at 300 RPM is $1 \times 10^{-5} \frac{m}{N}$.

Problem 1f

Figure 5 shows the displacement of the washing machine.

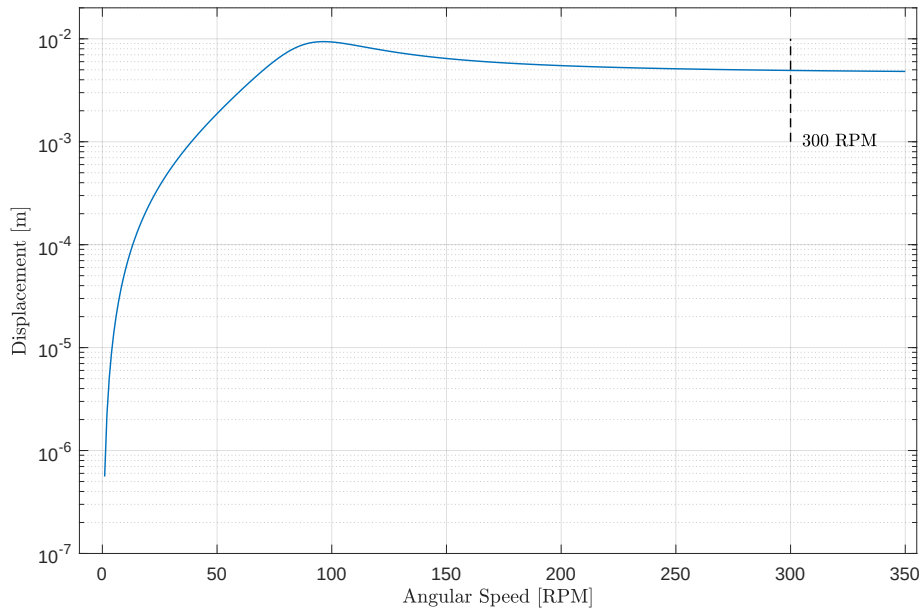


Figure 5: Displacement versus RPM.

The displacement of the washing machine at 300 RPM is approximately 4.9×10^{-3} m. The washing machine undergoes larger momentary displacements as it spins up to 300 RPM, in particular around 100 RPM.

Problem 2 - Washing Machine Mount Design (2 DOF)

The Matlab code for this problem is listed in Appendix 2.

Problem 2a

Figure 1 illustrates the two-stage vibration mount showing the washing machine and the “raft”.

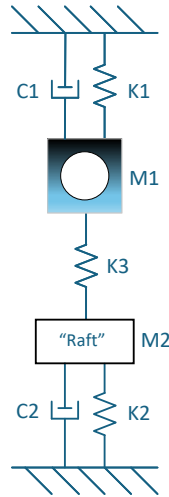


Figure 1: Two-stage vibration mount system for the washing machine.

Problem 2b

Table 1 lists the initial stiffness and damping coefficients for the system.

M1	110	kg
K1	100	$\frac{\text{N}}{\text{m}}$
C1	5	$\frac{\text{kg}}{\text{s}}$

M2	100	kg
K2	100	$\frac{\text{N}}{\text{m}}$
C2	5	$\frac{\text{kg}}{\text{s}}$

M3	0	kg
K3	1,000	$\frac{\text{N}}{\text{m}}$
C3	5	$\frac{\text{kg}}{\text{s}}$

Table 1: Initial values of stiffness and damping coefficients.

Problem 2c

Figure 2 illustrates the admittance of the washing machine and the raft with the force applied to the washing machine.

At the operating speed of 300 RPM, the admittance of the washing machine is $9.31 \times 10^{-6} \frac{\text{m}}{\text{N}}$.

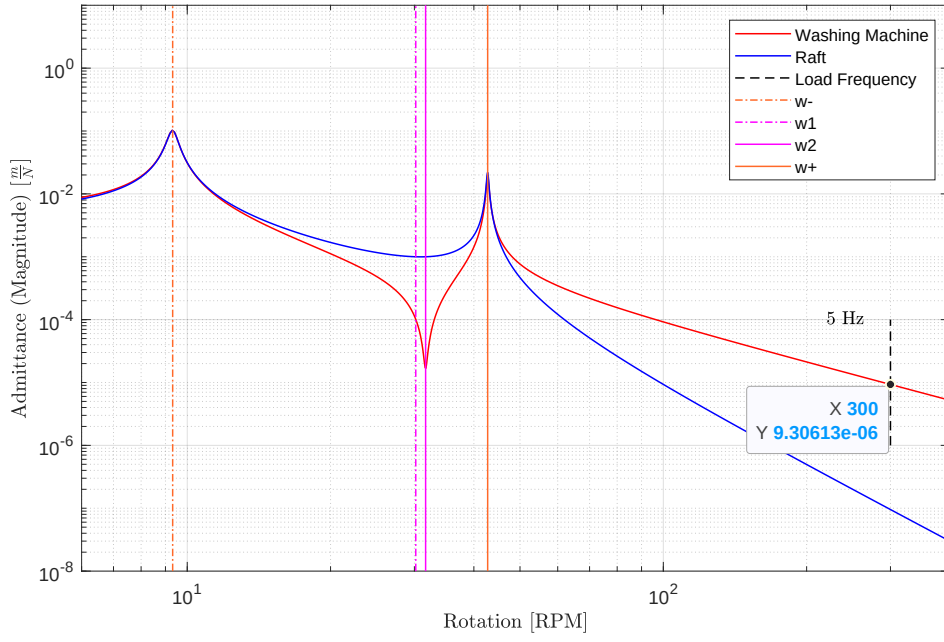


Figure 2: Admittance of the washing machine and the raft with a force on the washing machine.

Problem 2d

Figure 3 illustrates the displacement of the washing machine and the raft with the force applied to the washing machine.

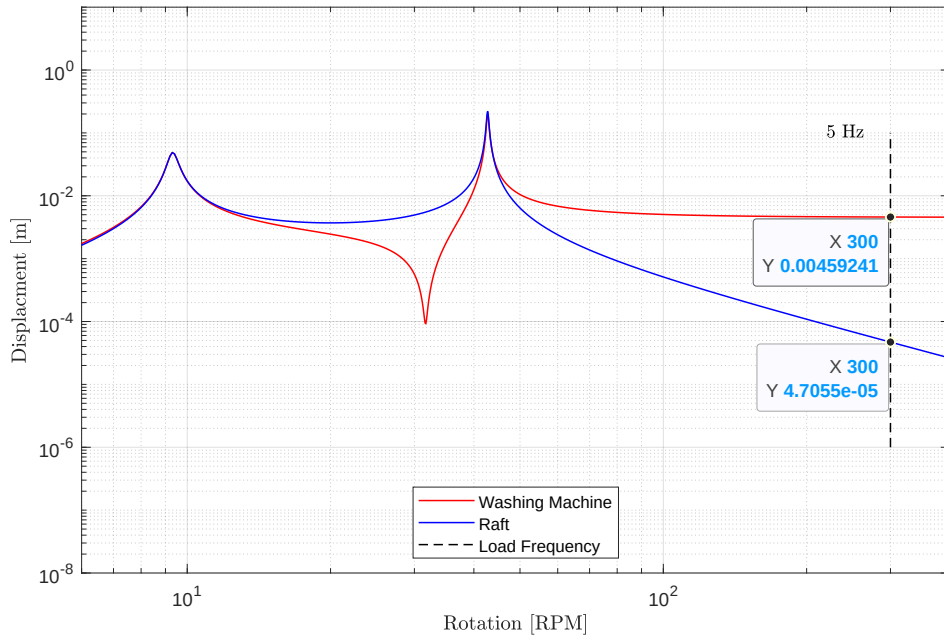


Figure 3: Displacement of the washing machine and the raft with a force on the washing machine.

At the operating speed of 300 RPM, the displacement of the washing machine is 4.6×10^{-3} m, which is less than the 5.0×10^{-3} m displacement with the first-order system.

The displacement of the raft is 4.7×10^{-5} m.

Problem 2e

Figure 4 illustrates the force applied to the ground by the raft. This is calculated by multiplying the displacement of the raft by the K2 spring constant.

The load on the ground is always less than 100 N.

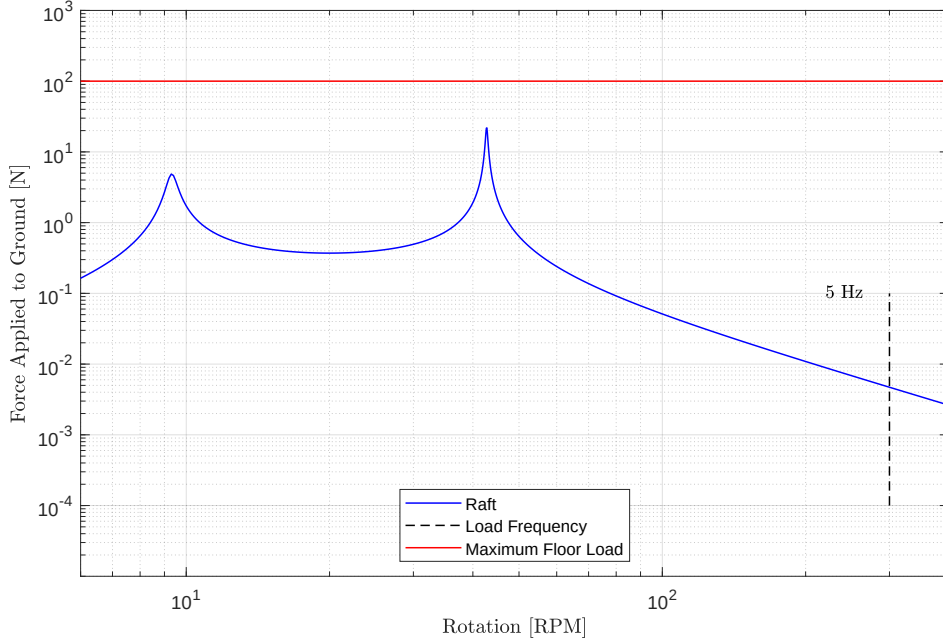


Figure 4: Force applied to the ground by the raft.

Problem 2f

The initial parameter set chosen, see Table ??, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 2g

The initial parameter set chosen, see Table ??, facilitated a displacement of the washing machine of 4.6×10^{-3} m at 300 RPM, which is less than the 5.0×10^{-3} m found for the 1 DOF system. The force applied to the ground by the raft is less than the required 100 N maximum.

Problem 3 - Washing Machine Dynamic Vibration Absorber Design

The Matlab code for this problem is listed in Appendix 3.

Part 3a

Figure 1 illustrates the dynamic vibration absorber (DVA) system.

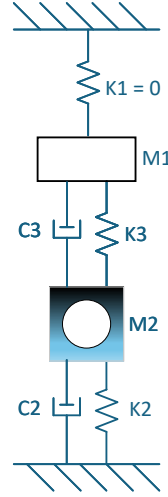


Figure 1: Dynamic vibration absorber system for the washing machine.

Table 2 lists the stiffness and damping coefficients for the system.

M1	15	kg
K1	0	$\frac{\text{N}}{\text{m}}$
C1	0	$\frac{\text{kg}}{\text{s}}$

M2	110	kg
K2	9,810	$\frac{\text{N}}{\text{m}}$
C2	1,962	$\frac{\text{kg}}{\text{s}}$ ¹

M3	0	kg
K3	1e5	$\frac{\text{N}}{\text{m}}$
C3	1e4	$\frac{\text{kg}}{\text{s}}$ ²

Table 2: Stiffness and damping coefficients for the DVA system.

Damping was implemented by adding with an imaginary component to the respective spring stiffness³.

¹0.2 fraction of respective spring stiffness.

²0.1 fraction of respective spring stiffness.

³Not equivalent to using a damping vector in the argument to the given Matlab nDOF function script-file.

Part 3b

Figure 2 shows the admittance response of the original system and DVA system based on the equations presented in the tutorial lecture on DVA system analysis.

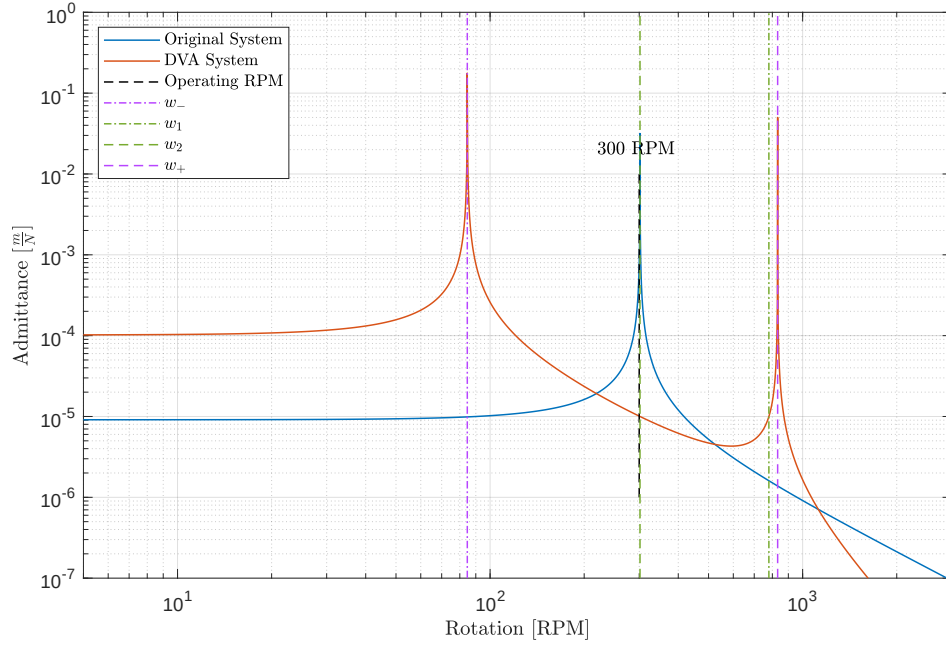


Figure 2: Admittance responses based on the equations from the DVA tutorial lecture.

The washing machine admittance at 300 RPM is reduced from the blue curve resonance peak to a point between the two resonances on the red curve).

Part 3c - Admittance

Figure 3 shows the admittance response of the DVA system using the nDOF function script-file. The stiffness of the coupling spring, k_3 , is set to place the null of the admittance response at the 300 RPM operating state of the washing machine. *The admittance at 300 RPM is approximately $1.2 \times 10^{-6} \frac{m}{N}$.*

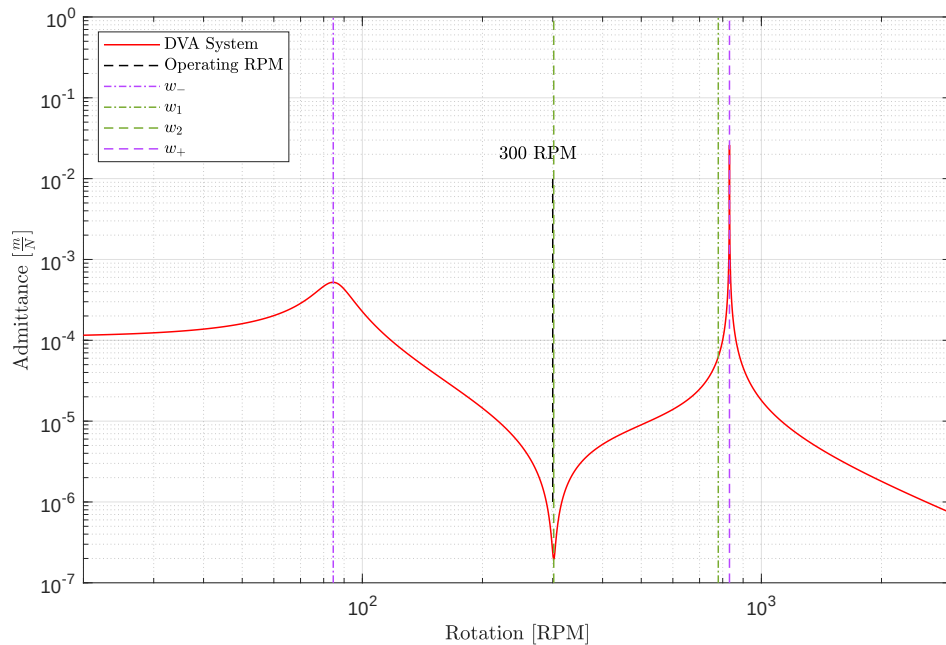


Figure 3: Solution using nDOF script-file function.

Part 3d - Displacement

Figure 4 shows the displacement of the washing machine as a function of RPM. The ramp up to 300 RPM occurs quickly so that the large displacements are transient and the maximum speed is 350 RPM (the large displacement associated with the resonance peak at higher frequencies is not reached). *The displacement at 300 RPM is approximately 38.2×10^{-6} m.*

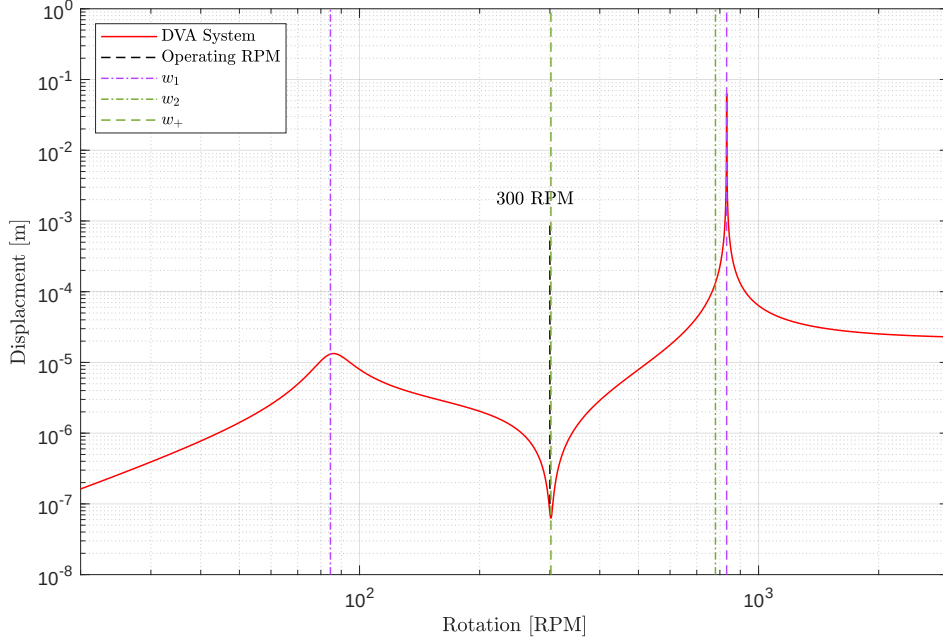


Figure 4: Displacement of the washing machine.

Part 3e

Figure 5 shows the force applied to the floor by the washing machine. *The force applied to the floor at 300 RPM is approximately 3.75×10^{-3} m.*

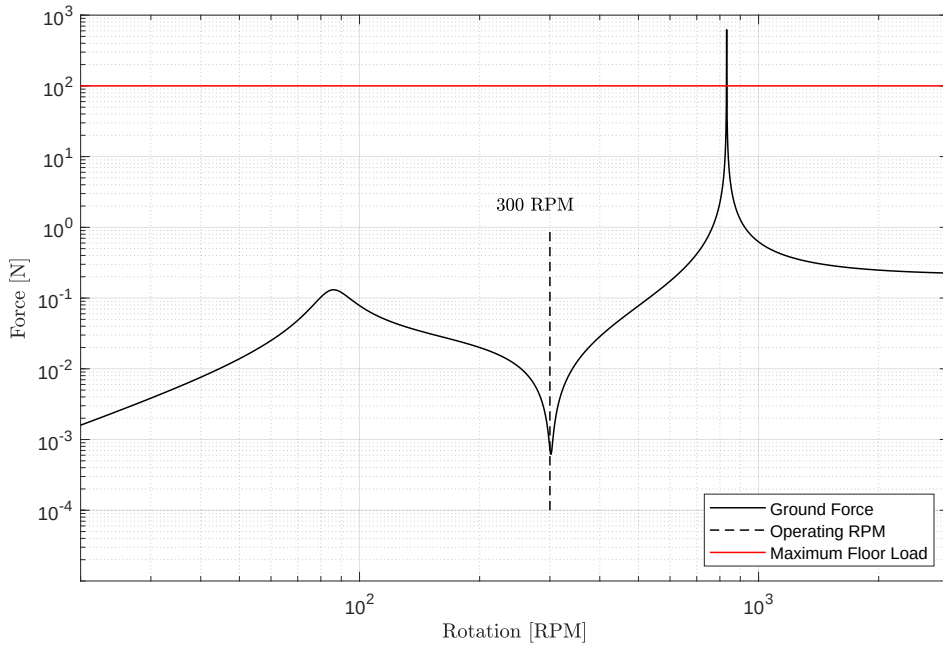


Figure 5: Force applied to the floor.

Problem 4 - Washing Machine Abuse Testing

The Matlab code for this problem is listed in Appendix [4](#).

Part 4a

Summary

ph

1 Appendix - Matlab Code for Problem 1

```
1
2
3
4 %% Synopsis
5
6 % Problem 1 – Washing Machine Mount Design – 1 Degree of Freedom (DOF)
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( '../00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowSize', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format LongG;
23
24 pause( 1 );
25
26
27
28 %% Anonymous Function Definitions
29
30 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
    load_distance;
31
32 h_transmissibility = @( epsilon, r ) sqrt( ( 1 + (2.*epsilon.*r).^2 ) ./ ( ( 1 - r.^2 ).^2 + (
    2.*epsilon.*r ).^2 ) );
33
34 rpm_conversion_to_radians_per_second = 0.10472;
35
36
37
38 %% Washing Machine Information
39
40 machine.mass = 100; % kg
41 machine.rotation_speed = 31.4; % radians\s or 300 rotations per minute (RPM)
42 machine.load_mass = 10; % kg
43 machine.static_displacement = 1e-2; % m
44
45
46
47 %% Load Force Information
48
49 dynamic_force.mass = 1; % kg
50 dynamic_force.radius = 0.5; % m
51
52 force_frequency = 300 * rpm_conversion_to_radians_per_second / ( 2 * pi ); % 5 Hz
53
54 % Maximum force applied to foundation is 100 N.
55
56
57
58 %% Part 1a
59
60 rpm = 0:1:350;
61 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
62
63
64 figure( 'Name', 'Problem 1a – Load Force Versus Angular Velocity' ); ...
65 plot( rpm, h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ) ); hold on;
66 plot( 300, 494, 'Marker', '.', 'MarkerSize', 20 );
67 line( [ 300, 300 ], [ 400 600 ], 'Color', 'r' );
68 text( 305, 494, '493.5 N' ); grid on;
69 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Load Force [N]' );
70 axis( [ -10 355 -5 705 ] );
71
72
73
```

```

74 %% Part 1b
75
76 maximum_allowable_displacement = 1e-2; % m
77 ks = ( machine.load_mass * 9.81) / maximum_allowable_displacement; % 9,810 N\m
78
79
80 total_mass = machine.mass + machine.load_mass; % 110 kg
81 wo = sqrt( ks / total_mass ); % 9.4 radians\s
82 fo = wo / (2*pi); % 1.5 Hz - Natural frequency of the mount.
83
84
85
86 %% Part 1c
87
88 frequency_set = 0.1:0.01:1e3;
89 r = frequency_set ./ fo;
90
91 damping_ratio = logspace( log10(0.001), log10(1), 6 );
92
93
94 figure( 'Name', 'Problem 1c - Transmission Loss' ); ...
95 hold on;
96 %
97 for index = 1:1:numel( damping_ratio )
98     plot( r, 10*log10( h_transmissibility( damping_ratio( index ), r ) ) );
99 end
100 %
101 xlabel( 'Frequency Ratio,  $\frac{f}{f_0}$ ' ); ylabel( 'Transmission Loss [dB]' );
102 legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'NorthEast' );
103 axis( [ 0.1 665 -60 25 ] );
104 grid on; box on;
105 set( gca, 'XScale', 'log' );
106
107
108
109 %% Part 1d
110
111 rpm = 0:1:350; % rotations per minute
112 rpm_conversion_to_radians_per_second = 0.10472; % radians\s
113 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
114
115 frequency_set = angular_velocity / (2*pi);
116 r = frequency_set ./ fo;
117
118
119 figure( 'Name', 'Problem 1d - Applied Force to Foundation' ); ...
120 hold on;
121 %
122 for index = 1:1:numel( damping_ratio )
123     plot( rpm, h_transmissibility( damping_ratio( index ), r ) .* h_force( dynamic_force.mass
124         , angular_velocity, dynamic_force.radius ) );
125 end
126 %
127 line( [ 300, 300 ], [ 2 1e4 ], 'Color', 'k', 'LineStyle', '—' );
128 text( 305, 2, '300 RPM' ); grid on;
129 line( [ 0 350 ], [ 100 100 ], 'Color', 'r', 'LineStyle', '—' );
130 text( 0, 150, '100 N' ); grid on;
131 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Force Applied to Foundation [N]' );
132 legend( '0.001', '0.004', '0.015', '0.063', '0.25', '1', 'Location', 'South' );
133 axis( [ -10 355 1 2e4 ] );
134 grid on; box on;
135 set( gca, 'YScale', 'log' );
136
137
138 %% Part 1e
139
140 % The best damping ratio is 0.25.
141
142 C = 0.25 * 2*sqrt( ks * total_mass ); % 519.4 kg\s
143
144 % The units of a viscous damping coefficient are Newton-seconds per meter (Ns/m) or kilograms per
145 % second (kg/s).
146
147 m = total_mass;
148 k = [ ks 0 ];
149 dampings = [ C 0 ];

```

```

150     admittance = nDOF_direct_solution( m, k, dampings, frequency_set, 'admittance' );
151
152
153 figure( 'Name', 'Problem 1e - Admittance' ); ...
154 semilogy( rpm, abs( admittance ) ); grid on;
155 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
156 set( gca, 'YScale', 'log' );
157 %
158 line( [ 300, 300 ], [ 7e-6 2e-5 ], 'Color', 'k', 'LineStyle', '—' );
159 text( 305, 2e-5, '300 RPM' ); grid on;
160 axis( [ -10 355 6e-6 2.5e-4 ] );
161
162
163 % temp2 = abs( admittance );
164 % round( temp2( 301 ), 3, 'significant' )
165
166
167
168 %% Part 1f
169
170 displacement = abs( admittance ) .* h_force( dynamic_force.mass, angular_velocity, dynamic_force.
    radius ).';
171
172
173 figure( 'Name', 'Displacement' ); ...
174 plot( rpm, displacement ); grid on;
175 xlabel( 'Angular Speed [RPM]' ); ylabel( 'Displacement [m]' );
176 line( [ 300, 300 ], [ 1e-3 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
177 text( 305, 1e-3, '300 RPM' ); grid on;
178 set( gca, 'YScale', 'log' );
179 axis( [ -10 355 1e-7 2e-2 ] );
180
181
182 temp2 = abs( displacement );
183 round( temp2( 301 ), 3, 'significant' )
184
185
186
187 %% Clean-up
188
189 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
190     monitors = get( 0, 'MonitorPositions' );
191     if ( size( monitors, 1 ) == 1 )
192         autoArrangeFigures( 3, 4, 1 );
193     elseif ( 1 < size( monitors, 1 ) )
194         autoArrangeFigures( 2, 2, 2 );
195     end
196 end
197
198 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
199
200
201
202 %% Reference(s)

```

2 Appendix - Matlab Code for Problem 2

```
1
2
3
4 %% Synopsis
5
6 % Problem 2 — Washing Machine Two-stage Mount Design
7
8 % See Lecture 14, Monday, March 3, 2025
9
10
11
12 %% Note(s)
13
14 % A 2 degree-of-freedom (DOF) system will provide isolation performance 1 DOF system.
15
16 % Allowing the top and bottom masses, the washing machine and the raft ,
17 % respectively , to each be attached to ground independently.
18
19 % With the force applied to the washing machine and the displacement of the
20 % washing machine, use FSF( :, 1, 1 ). However, if the force is applied elsewhere ,
21 % than FSF( :, 2, 2, ) must be used.
22
23
24
25 %% Environment
26
27 close all; clear; clc;
28 % restoredefaultpath;
29
30 addpath( genpath( './00 Support' ), '-begin' );
31
32 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
33 set( 0, 'DefaultFigureWindowStyle', 'normal' );
34 set( 0, 'DefaultLineLineWidth', 0.8 );
35 set( 0, 'DefaultTextInterpreter', 'Latex' );
36
37 format ShortG;
38
39 pause( 1 );
40
41
42
43 %% Define Anonymous Functions
44
45 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
46
47 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
48 h_w_to_f = @( w ) w/(2*pi);
49
50 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
51
52
53
54 %% Problem 2a
55
56 % See report .
57
58
59
60 %% Problem 2b — Blocked Frequencies of Resonance
61
62 % The frequency of the forcing function is 5 Hz and the amplitude is 493.5 N.
63 fo = 5; % Hz
64 wo = 2*pi*fo; % 31.4 radians\s
65
66
67 m1 = 100 + 10; % kg — Total mass of the washing machine and the load.
68 k1 = 1e2; % N\m
69 c1 = 5;
70
71 m2 = 100; % kg — UPPER LIMIT OF 100 kg
72 k2 = 1e2; % N\m
73 c2 = 5;
74
75 k3 = 1e3; % N\m -> Larger values produce stronger coupling (higher w+).
```

```

76
77
78 w1 = sqrt( ( k1 + k3 ) / m1 ); % 3.2 radians\s; washing machine
79 f1 = w1 / (2*pi); % 0.5 Hz
80 rpm1 = f1*(2*pi)/0.10471; % 30.1 RPM
81
82 w2 = sqrt( ( k2 + k3 ) / m2 ); % 3.3 radians\s; raft
83 f2 = w2 / (2*pi); % 0.53 Hz
84 rpm2 = f2*(2*pi)/0.10471; % 31.7 RPM
85
86 % Note: If k3 is set to zero, then these frequencies represent the
87 % resonance frequencies for each mass on their own.
88
89
90
91 %% Problem 2b — Coupled Frequencies
92
93 mu_4 = ( k3^2 / (m1*m2) ); % 90.9 (radians\s)^4
94 mu = ( mu_4 ) ^0.25; % 3.1 radians\s
95
96 w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
97 w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
98 w = sqrt( w );
99
100
101 w_minus = w(2); % 0.97 radians\s (lower than w1, 1, and w2, 1.05 radians\s)
102 f_minus = w(2)/(2*pi); % 0.1545 Hz
103 rpm_minus = h_f_to_rpm( f_minus ); % 9.3 RPM
104 %
105 w_plus = w(1); % 4.5 radians\s (higher than w1, 1, and w2, 1.05 radians\s)
106 f_plus = w(1)/(2*pi); % 0.71 Hz
107 rpm_plus = h_f_to_rpm( f_plus ); % 42.8 RPM
108 %
109 % Note(s):
110 %
111 % The blocked frequencies, w1 and w2, are always between these two frequencies.
112
113 clear w mu_4;
114
115
116
117 %% Problem 2b — Mode Shapes
118
119 phi_plus = [ ...
120 1; ...
121 -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
122 ];
123 %
124 % 1
125 % -1.10
126
127 phi_minus = [ ...
128 1; ...
129 (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
130 ];
131 %
132 % 1
133 % 0.99
134
135
136 % Note(s):
137 %
138 % When m1 = m2 and k1 = k3,
139 %
140 % a.) At w_minus, m1 and m2 move the same amount and are in-phase.
141 % b.) At w_plus, m1 and m2 move the same amount, but are out-of-phase.
142
143 % In general, for a 2 DOF system there will be 2 modes (in-phase and out-of-phase).
144
145 % With different initial conditions (no forcing), the behaviour is a weighted sum of the two
146 % modes.
147
148
149 %% Problem 2c
150
151 masses = [ m1 m2 ];
152 stiffnesses = [ k1 k3 k2 ];

```



```

153 dampings = [ c1  0  c2 ];
154
155 % ADD DAMPING AS A COMPLEX COMPONENT TO THE SPRING STIFFNESS.
156
157
158 rpm = 0:0.1:500; % rotations per minute
159 rpm_conversion_to_radians_per_second = 0.10472;
160 angular_velocity = rpm .* rpm_conversion_to_radians_per_second; % radians\s
161 frequencies = angular_velocity / (2*pi); % Hz
162
163 FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' );
164
165
166 admittance = zeros( numel( frequencies ), 1 );
167
168 for index = 1:1:numel( frequencies )
169     temp = diag( squeeze( FRF( index, 1, 1 ) ) ); % Check indexing to ensure force on washing
170     admittance( index ) = abs( temp );
171 end
172 %
173 clear temp temp2;
174
175 washing_machine.admittance = admittance;
176
177
178 admittance = zeros( numel( frequencies ), 1 );
179
180 for index = 1:1:numel( frequencies )
181     temp = diag( squeeze( FRF( index, 1, 2 ) ) ); % Check indexing to ensure force on washing
182     admittance( index ) = abs( temp );
183 end
184 %
185 clear temp temp2;
186
187 raft.admittance = admittance;
188
189
190 figure( 'Name', 'Admittance - Magnitude' ); ...
191 h1 = loglog( rpm, washing_machine.admittance, 'Color', 'r' ); hold on;
192 h2 = loglog( rpm, raft.admittance, 'Color', 'b' );
193 h3 = line( [ 300 300 ], [ 1e-6 1e-4 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
194 text( 220, 1e-4, '5 Hz' );
195 %
196 h4 = line( [ rpm_minus rpm_minus ], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle',
197     '—.' );
198 %
199 h5 = line( [ rpm1 rpm1 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '—.' );
200 h6 = line( [ rpm2 rpm2 ], [ 1e-8 1e1 ], 'Color', 'm', 'LineStyle', '—' );
201 h7 = line( [ rpm_plus rpm_plus ], [ 1e-8 1e1 ], 'Color', [ 1.00, 0.41, 0.16 ], 'LineStyle',
202     '—.' );
203 %
204 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Washing Machine', 'Raft', 'Load Frequency', 'w—', 'w1', 'w2', 'w+', 'Location', 'NorthEast' );
205 %
206 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance (Magnitude) [ $\frac{m}{N}$ ]' );
207 axis( [ 6 400 1e-8 1e1 ] );
208
209 % temp2 = abs( FRF( :, 1, 1 ) );
210 % round( temp2( 3001 ), 3, 'significant' )
211
212 return
213
214 %% Problem 2d
215
216 dynamic_force.mass = 1; % kg
217 dynamic_force.radius = 0.5; % m
218
219 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
220     load_distance;
221
222 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
223
224 figure( 'Name', '' ); ...

```

```

225     h1 = loglog( rpm, washing_machine.admittance.*temp, 'Color', 'r' ); hold on;
226     h2 = loglog( rpm, raft.admittance.*temp, 'Color', 'b' );
227     h3 = line( [ 300 300 ], [ 1e-6 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
228     text( 220, 1e-1, '5 Hz' );
229     legend( [ h1 h2 h3 ], 'Washing Machine', 'Raft', 'Load Frequency', 'Location', 'South' );
230     xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
231     axis( [ 6 400 1e-8 1e1 ] );
232
233
234
235 %% Problem 2e
236
237 % Displacement of the raft multiplied by the K2 spring constant.
238
239 dynamic_force.mass = 1; % kg
240 dynamic_force.radius = 0.5; % m
241
242 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
    load_distance;
243
244 temp = h_force( dynamic_force.mass, angular_velocity, dynamic_force.radius ).';
245
246
247 figure( 'Name', 'Force Applied to Ground by Raft' ); ...
248     h1 = loglog( rpm, raft.admittance.*temp*k2, 'Color', 'b' ); hold on;
249     h2 = line( [ 300 300 ], [ 1e-4 1e-1 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
250     h3 = line( [ 6 400 ], [ 100 100 ], 'Color', 'r' );
251     text( 220, 1e-1, '5 Hz' );
252     legend( [ h1 h2 h3 ], 'Raft', 'Load Frequency', 'Maximum Floor Load', 'Location', 'South'
    );
253     xlabel( 'Rotation [RPM]' ); ylabel( 'Force Applied to Ground [N]' );
254     axis( [ 6 400 1e-5 1e3 ] );
255
256
257
258 %% Problem 2f
259
260 % return
261
262 %% Problem 2g
263
264 % return
265
266 %% 2 DOF Example
267
268 % masses = [ 50 1 ];
269 % stiffnesses = [ 100e3 1800 0 ];
270 %
271 % dampings = [ 0 0 0 ];
272 % dampings = [ 0 (1 + 0.1*1j) (1 + 0.1*1j) ];
273 %
274 % frequencies = 0:0.01:40;
275 %
276 % FRF = nDOF_direct_solution( masses, stiffnesses, dampings, frequencies, 'admittance' ); %
    4001-by-2-by-2
277 %
278 %
279 % admittance = zeros( numel( frequencies ), 1 );
280 %
281 % for index = 1:1:numel( frequencies )
282 %     temp = diag( squeeze( FRF( index, :, : ) ) );
283 %     temp2 = temp(1) + temp(2)*1j;
284 %     admittance( index ) = abs( temp2 );
285 % end
286 %
287 % clear temp1 temp2;
288 %
289 %
290 %% figure( ); ...
291 %% semilogy( frequencies, admittance ); grid on;
292 %% xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
293 %
294 %
295 % figure( 'Name', 'Admittance' ); ...
296 % semilogy( frequencies, abs( FRF( :, 1, 1 ) ) ); hold on;
297 % semilogy( frequencies, abs( FRF( :, 1, 2 ) ) );
298 % semilogy( frequencies, abs( FRF( :, 2, 1 ) ) );
299 % semilogy( frequencies, abs( FRF( :, 2, 2 ) ) ); grid on;

```

```

300 %         legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
Movement with F2', 'Location', 'North' );
301 %         xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
302 %         axis( [ 5.5 10.5 1e-7 1e2 ] );
303
304
305 % figure( 'Name', 'Dynamic Stiffness (Reciprocal of Admittance)' ); ...
306 %         semilogy( frequencies, 1 ./ abs( FRF( :, 1, 1 ) ) ); hold on;
307 %         semilogy( frequencies, 1 ./ abs( FRF( :, 1, 2 ) ) );
308 %         semilogy( frequencies, 1 ./ abs( FRF( :, 2, 1 ) ) );
309 %         semilogy( frequencies, 1 ./ abs( FRF( :, 2, 2 ) ) ); grid on;
310 %         legend( 'M1 Movement with F1', 'M1 Movement with F2', 'M2 Movement with F1', 'M2
Movement with F2', 'Location', 'North' );
311 %         xlabel( 'Frequency [Hz]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
312
313
314
315 %% Clean-up
316
317 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
318     monitors = get( 0, 'MonitorPositions' );
319     if ( size( monitors, 1 ) == 1 )
320         autoArrangeFigures( 3, 4, 1 );
321     elseif ( 1 < size( monitors, 1 ) )
322         autoArrangeFigures( 2, 2, 2 );
323     end
324 end
325
326 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
327
328
329
330 %% Reference(s)
331
332
333
334 %% Comment(s)
335
336 %% Regions of Interest
337
338 % 5 regions of interest:
339
340 % 1.) Low frequency:  $w < w_{\text{minus}}$ :
341 %
342 % Stiffness controlled; two masses moving together; strong coupling ( $k_3$  high).
343
344
345 % 2.) Force at a resonance frequency:  $w = w_{\text{minus}}$  (lowest resonance frequency):
346 %
347 % At lowest resonance frequency; at  $w_{\text{minus}}$  mode, masses moving in same direction (in-phase)
with same high amplitude
348
349
350 % 3.) Forcing frequency between  $w_{\text{minus}}$  and  $w_2$ 
351
352
353 % 4.) Forcing frequency  $w = w_2$ 
354
355
356 % 5.) Forcing frequency  $w < w < w_+$ 
357 %
358 % No a special frequency.
359
360
361 % 6.) Forcing frequency  $w = w_+$ 
362 %
363 % At highest resonance frequency; at  $w_{\text{plus}}$  mode, masses moving in opposite directions (out-of-
phase) with same high amplitude
364
365
366 % 7.) Forcing frequency  $w_+ < w$ 
367 %
368 % Mass controlled region; mass 1 (washing machine) moves more than mass 2 (raft);

```

3 Appendix - Matlab Code for Problem 3

```
1
2
3 % To do:
4
5 % remove comment statements
6
7 % verify indices for required values
8
9
10
11 %% Synopsis
12
13 % Problem 3 – Washing Machine Dynamic Vibration Absorber (DVA) Design
14
15
16
17 %% Note(s)
18
19 % At 300 RPM, the load frequency, this system will isolated vibration more
20 % than the 2 DOF and 1 DOF systems.
21
22 % The DVA behaves like a Helmholtz resonator, working as a mechanical notch
23 % filter at a particular frequency.
24
25 % DVAs work well in systems that have one dominate mode. A good
26 % example of this is building sway and its compensation. Based on the
27 % design and construction of a building, it will typically have a single,
28 % dominate mode.
29
30 % DVA is useful if you can not get away from the resonance frequency; can
31 % not tune a 2 DOF system when the resonance frequency is away from the
32 % forcing frequency.
33
34 % DVA introduces two resonances for the original resonance. These new
35 % resonances will produce greater motion at the new resonance frequencies.
36
37
38
39 %% Environment
40
41 close all; clear; clc;
42 % restoredefaultpath;
43
44 addpath( genpath( './00 Support' ), '-begin' );
45
46 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
47 set( 0, 'DefaultFigureWindowSize', 'normal' );
48 set( 0, 'DefaultLineLineWidth', 0.8 );
49 set( 0, 'DefaultTextInterpreter', 'Latex' );
50
51 format ShortG;
52
53 pause( 1 );
54
55
56
57 %% Define Anonymous Functions
58
59 h_rpm_to_f = @( rpm ) ( rpm * 0.10471 ) / ( 2 * pi );
60 h_rpm_to_w = @( rpm ) ( rpm * 0.10471 );
61
62 h_f_to_rpm = @( f ) ( f*2*pi ) / 0.10471;
63
64 h_w_to_f = @( w ) w/(2*pi);
65 h_w_to_rpm = @( w ) h_f_to_rpm( w*2*pi );
66
67
68 h_force = @( force_mass, rotation_speed, load_distance ) force_mass .* rotation_speed.^2 .*
        load_distance;
69
70
71
72 %% Washing Machine Information
73
74 machine.mass = 100; % kg
```

```

75 machine.rotation_speed = 31.4; % radians\s or 300 rotations per minute (RPM)
76 machine.load_mass = 10; % kg
77 machine.static_displacement = 1e-2; % m
78
79
80
81 %% Load Force Information
82
83 dynamic_force.mass = 1; % kg
84 dynamic_force.radius = 0.5; % m
85
86 % Maximum force applied to foundation is 100 N.
87
88
89
90 %% Dynamic Vibration Absorber (DVA)
91
92 maximum_allowable_displacement = 1e-2; % m
93 ks = ( machine.load_mass * 9.81 ) / maximum_allowable_displacement; % 9,810 N\m
94
95
96 m = 110; % kg; washing machine mass and load mass
97 k = ks; % 9,810 N\m
98
99 wo = sqrt( k / m ); % 9.4 radians\s
100 fo = wo/(2*pi); % 1.5 Hz
101
102
103 m_dva = 15; % kg
104 k_dva = 1e5; % N\m
105
106 wo_dva = sqrt( k_dva / m_dva ); % 81.7 radians\s
107 fo_dva = wo_dva/(2*pi); % 13.0 Hz
108
109
110
111 %% Blocked Frequencies (Equation 9.1 of the Notes)
112
113 m1 = m_dva;
114 k1 = 0; % Spring above DVA mass; does not exist; set to zero.
115
116 m2 = m;
117 k2 = k;
118
119 k3 = k_dva;
120
121
122 w1 = sqrt( ( k1 + k3 ) / m1 ); % 81.7 radians\s; smaller than the system resonant frequency.
123 f1 = w1/(2*pi); % 13 Hz
124
125 w2 = sqrt( ( k2 + k3 ) / m2 ); % 31.6 radians\s; greater than the system resonant frequency.
126 f2 = w2/(2*pi); % 5.0 Hz
127
128
129
130 %% Coupled Frequencies (Equation 9.15 of the Notes)
131
132 mu_4 = ( k3^2 / (m1*m2) ); % 6.1 (kg\s)^4
133 mu = ( mu_4 )^0.25; % 49.1 kg\s
134
135 w(1) = 0.5*( ( w1^2 + w2^2 ) + sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
136 w(2) = 0.5*( ( w1^2 + w2^2 ) - sqrt( ( w1^2 - w2^2 )^2 + 4*mu_4 ) );
137 w = sqrt( w );
138
139
140 w_minus = w(2); % 8.9 radians\s (lower than 31.6 and 81.7 radians\s)
141 f_minus = w(2)/(2*pi); % 1.4 Hz
142 %
143 w_plus = w(1); % 87.1 radians\s (greater than 31.6 and 81.7 radians\s)
144 f_plus = w(1)/(2*pi); % 13.9 Hz
145
146
147 % The blocked frequencies are always between the lower and higher coupled frequencies.
148
149
150
151 %% Mode Shapes (Equations 9.18 and 9.19 of the Notes)
152

```

```

153 phi_plus = [ ...
154     1; ...
155     -(1/mu^2)*sqrt(m1/m2)*(w_plus^2 - w1^2), ...
156 ];
157 %
158 % 1
159 % -0.14
160
161
162 phi_minus = [ ...
163     1; ...
164     (1/mu^2)*sqrt(m1/m2)*(w1^2 - w_minus^2), ...
165 ];
166 %
167 % 1
168 % 0.99
169
170
171
172 %% Admittance Using Equations from DVA Lecture
173
174 MAXIMUM_RPM = 3e3;
175
176 F0 = 1;
177
178 w = 0:0.01:h_rpm_to_w( MAXIMUM_RPM );
179 f = w/(2*pi);
180 rpm = h_f_to_rpm( f );
181
182
183 m = [ 15 110 ];
184 k = [ 0 k_dva.*(1+0.1*1i) k2.*(1+0.2*1i) ];
185
186
187 admittance_modified = F0 ./ ( m(1)*m(2) )*k(2) ./ (( w.^2 - w_plus^2 ) .* ( w.^2 - w_minus^2 ));
188 admittance_original = F0 ./ m(2) * 1 ./ ( w2^2 - w.^2 );
189
190
191 figure( 'Name', 'Admittance - Equations from DVA Lecture' ); ...
192 h1 = loglog( rpm, abs( admittance_original ) ); hold on;
193 h2 = loglog( rpm, abs( admittance_modified ) );
194 h3 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
195 text( 220, 2e-2, '300 RPM' );
196 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
197     0.27, 1.00 ], 'LineStyle', '-.' );
198
199 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
200     ], 'LineStyle', '-.' );
201 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
202     ], 'LineStyle', '—' );
203
204 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
205     0.27, 1.00 ], 'LineStyle', '—' ); grid on;
206
207 axis( [ 5 3e3 1e-7 1 ] );
208 legend( [ h1 h2 h3 h4 h5 h6 h7 ], 'Original System', 'DVA System', 'Operating RPM', ...
209     '$w_-$', '$w_1$', ...
210     '$w_2$', '$w_+$', ...
211     'Location', 'NorthWest', 'Interpreter', 'Latex' );
212 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [$\frac{m}{N}$]' );
213
214
215 %% Admittance Using nDOF Function
216
217 % Damping can be added by using a dampings vector or appending a complex
218 % value to the respective spring stiffness. THESE ARE NOT INTERCHANGABLE.
219
220 rpm = 0:0.1:MAXIMUM_RPM;
221 f = h_rpm_to_f( rpm );
222 w = f ./ (2*pi);
223
224 FRF = nDOF_direct_solution( m, k, [ 0 0 0 ], f, 'admittance' ); % Symmetric matrix.
225 %
226 squeeze( FRF( numel( f ), :, : ) );
227 %
228 % (1, 1) - Force on M1 and its associated displacement.

```

```

227 % (2, 2) – Force on M2 and its associated displacement.
228 %
229 % (1, 2) – Force on M1 and the displacement of M2.
230 % (2, 1) – Force on M2 and the displacement on M1.
231 %
232 % These are interchangeable; the same result.
233
234
235 figure( 'Name', 'Admittance – nDOF Calculated' ); ...
236 h1 = loglog( h_f_to_rpm( f ) , abs( FRF( :, 1, 1 ) ) , 'Color', 'r' ); hold on;
237 h2 = line( [ 300 300 ], [ 1e-6 1e-2 ], 'Color', 'k', 'LineStyle', '—' );
238 text( 220, 2e-2, '300 RPM' );
239 h3 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—.' );
240
241 h4 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—.' );
242 h5 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-7 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—' );
243
244 h6 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-7 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—' ); grid on;
245
246 axis( [ 2e1 MAXIMUM_RPM 1e-7 1 ] );
247 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
248 '$w_{-}$', '$w_{1}$', ...
249 '$w_{2}$', '$w_{+}$', ...
250 'Location', 'NorthWest', 'Interpreter', 'Latex' );
251 xlabel( 'Rotation [RPM]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
252
253
254 % temp2 = abs( FRF( :, 1, 1 ) );
255 % round( temp2( 3001 ), 3, 'significant' )
256
257
258
259 %% Displacement
260
261 dynamic_force.mass = 1; % kg
262 dynamic_force.radius = 0.5; % m
263
264
265 figure( 'Name', 'Displacement' ); ...
266 h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
).', 'Color', 'r' ); hold on;
267 h2 = line( [ 300 300 ], [ 1e-7 1e-3 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
268 text( 220, 2e-3, '300 RPM' );
269 h4 = line( [ h_f_to_rpm( f_minus ) h_f_to_rpm( f_minus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—.' );
270
271 h5 = line( [ h_f_to_rpm( f1 ) h_f_to_rpm( f1 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—.' );
272 h6 = line( [ h_f_to_rpm( f2 ) h_f_to_rpm( f2 ) ], [ 1e-8 1e3 ], 'Color', [ 0.47, 0.67, 0.19
], 'LineStyle', '—' );
273
274 h7 = line( [ h_f_to_rpm( f_plus ) h_f_to_rpm( f_plus ) ], [ 1e-8 1e3 ], 'Color', [ 0.72,
0.27, 1.00 ], 'LineStyle', '—' ); grid on;
275
276 legend( [ h1 h2 h3 h4 h5 h6 ], 'DVA System', 'Operating RPM', ...
277 '$w_{-}$', '$w_{1}$', ...
278 '$w_{2}$', '$w_{+}$', ...
279 'Location', 'NorthWest', 'Interpreter', 'Latex' );
280
281 xlabel( 'Rotation [RPM]' ); ylabel( 'Displacement [m]' );
282 axis( [ 2e1 MAXIMUM_RPM 1e-8 1 ] );
283
284
285 % temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).';
286 % round( temp2( 3001 ), 3, 'significant' )
287
288
289
290 %% Ground Force
291
292 dynamic_force.mass = 1; % kg
293 dynamic_force.radius = 0.5; % m
294
295

```

```

296 figure( 'Name', 'Ground Force' ); ...
297 h1 = loglog( rpm, abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius
    ).'*k2, 'Color', 'k' ); hold on;
298 h2 = line( [ 300 300 ], [ 1e-4 1e-0 ], 'Color', 'k', 'LineStyle', '—' ); grid on;
299 text( 220, 2e-0, '300 RPM' );
300 h3 = line( [ 6 MAXIMUM_RPM ], [ 100 100 ], 'Color', 'r' );
301 legend( [ h1 h2 h3 ], 'Ground Force', 'Operating RPM', 'Maximum Floor Load', 'Location',
    'SouthEast' );
302 xlabel( 'Rotation [RPM]' ); ylabel( ' Force [N]' );
303 axis( [ 2e1 MAXIMUM_RPM 1e-5 1e3 ] );
304
305
306 temp2 = abs( FRF( :, 1, 1 ) ).*h_force( dynamic_force.mass, w, dynamic_force.radius ).'*k2;
307 % round( temp2( 3001 ), 3, 'significant' )
308
309
310
311 %% Clean-up
312
313 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
314     monitors = get( 0, 'MonitorPositions' );
315     if ( size( monitors, 1 ) == 1 )
316         autoArrangeFigures( 3, 4, 1 );
317     elseif ( 1 < size( monitors, 1 ) )
318         autoArrangeFigures( 2, 2, 2 );
319     end
320 end
321
322 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
323
324
325
326 %% Reference(s)

```


4 Appendix - Matlab Code for Problem 4

```
1
2
3
4 %% Synopsis
5
6 % Problem 4 – Washing Machine Abuse Testing
7
8
9
10 %% Environment
11
12 close all; clear; clc;
13 % restoredefaultpath;
14
15 addpath( genpath( './00 Support' ), '-begin' );
16
17 set( 0, 'DefaultFigurePaperPositionMode', 'manual' );
18 set( 0, 'DefaultFigureWindowSize', 'normal' );
19 set( 0, 'DefaultLineLineWidth', 0.8 );
20 set( 0, 'DefaultTextInterpreter', 'Latex' );
21
22 format ShortG;
23
24 pause( 1 );
25
26
27
28 %% Impulse Signal
29
30 time_step = 1e-4; % s
31 net_time = 10; % s
32 time_indices = 0:time_step:( net_time - time_step );
33
34 impulse = zeros( size( time_indices ) );
35 impulse( 4/time_step ) = 1./time_step;
36 brick_impulse = 5 .* impulse;
37 % brick_impulse = 50 .* impulse;
38
39
40 % figure( 'Name', 'Unit Impulse Signal' ); ... % Figure 1
41 %     plot( time_indices, impulse ); grid on;
42 %     legend( 'Unit Impulse Signal', 'Location', 'NorthEast' );
43 %     xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
44 %     axis( [ 0 net_time 0 125 ] );
45
46 % return
47
48 %% 1 DOF System Impulse Function
49
50 m = 5; % kg
51
52 % Initial conditions of 1 DOF system.
53 xo = 0; % m
54 vo = 1 / m; % m/s
55
56 wd = 31.4; % radians\s or 5 Hz or 300 RPM
57
58
59 ks = 9810; % N/m
60 wo = sqrt( ks / 110 ); % 9.4 radians\s
61 fo = wo / (2*pi); % 1.5 Hz
62
63
64 epsilon = 0.25;
65 C = epsilon * 2*sqrt( ks * 110 ); % 519.4 kg\s
66
67
68 h_unit_impulse = @( washing_machine_mass, wd, wo, epsilon, t ) ( 1./( washing_machine_mass.*wd)
69 ) .* exp( -wo.*epsilon.*t ) .* sin( wd.*t ); % mvo = 1 kg\ (m s)
70
71
72 %% 1 DOF System Impulse Response
73
74 impulse_response = h_unit_impulse( 100, wd, wo, epsilon, time_indices );
```

```

75 %
76 figure( 'Name', '1 DOF System Impulse Response' ); ... % Figure 2
77 plot( time_indices, impulse_response ); grid on;
78 xlabel( 'Time [s]' ); ylabel( 'Admittance [ $\frac{m}{N}$ ]' );
79 axis( [ -0.1 3 -2.5e-4 3.0e-4 ] );
80
81
82 response_to_brick_perturbation = conv( brick_impulse, impulse_response ) * time_step;
83 time_indices_perturbation = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
    time_step;
84 %
85 figure( 'Name', '1 DOF Impulse Response' ); ... % Figure 3
86 plot( time_indices_perturbation, response_to_brick_perturbation ); grid on;
87 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
88 axis( [ -0.1 5 -1.5e-3 1.5e-3 ] );
89
90
91
92 %% Sinusoidal Impulse Response
93
94 % sinusoid_input = 493.5 * sin( 2 * pi * 5 * time_indices );
95 sinusoid_input = 1 * sin( 2 * pi * 5 * time_indices );
96 % sinusoid_input = 493.5 * sin( 2 * pi * 1 * time_indices );
97 sinusoid_input( 1:1:( 1 / time_step ) ) = 0;
98 sinusoid_input( ( 8 / time_step ):1:( 10 / time_step ) ) = 0;
99 %
100 figure( 'Name', 'Sinusoidal Input' ); ... % Figure 4
101 plot( time_indices, sinusoid_input ); grid on;
102 xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
103 % xlim( [ 0 2.5 ] );
104 ylim( [ -600 600 ] );
105
106
107 response_to_sinusoidal_forcing = conv( sinusoid_input, impulse_response ) * time_step;
108 time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_sinusoidal_forcing ) - 1 ) ) .*
    time_step;
109 %
110 figure( 'Name', '1 DOF Sinusoidal Forcing Response' ); ... % Figure 5
111 plot( time_indices_sinusoidal_forcing, response_to_sinusoidal_forcing ); grid on;
112 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
113 % xlim( [ 0 2.5 ] );
114
115
116
117 %% Combined Response
118
119 joint_signal = sinusoid_input + brick_impulse;
120 %
121 figure( 'Name', 'Combined Signal' ); ... % Figure 6
122 plot( time_indices, joint_signal ); hold on;
123 plot( time_indices, sinusoid_input, 'LineStyle', '—' );
124 plot( time_indices, impulse, 'LineStyle', '—' );
125 xlabel( 'Time [s]' ); ylabel( 'Force [N]' );
126 % xlim( [ 0 2.5 ] );
127 % ylim( [ -600 600 ] );
128
129
130 response_to_combined_forcing = conv( impulse_response, joint_signal ) * time_step;
131 time_indices_sinusoidal_forcing = ( 0:1:( numel( response_to_brick_perturbation ) - 1 ) ) .*
    time_step;
132 %
133 figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
134 plot( time_indices_sinusoidal_forcing, response_to_combined_forcing ); grid on;
135 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
136 % xlim( [ 0 2.5 ] );
137
138
139
140 %% Plot Difference
141
142 figure( 'Name', '1 DOF Combined Forcing Response' ); ... % Figure 7
143 plot( time_indices_sinusoidal_forcing, response_to_combined_forcing -
    response_to_sinusoidal_forcing ); grid on;
144 xlabel( 'Time [s]' ); ylabel( 'Displacement [m]' );
145 % xlim( [ 0 2.5 ] );
146
147
148

```

```
149 %% Clean-up
150
151 if ( ~isempty( findobj( 'Type', 'figure' ) ) )
152     monitors = get( 0, 'MonitorPositions' );
153     if ( size( monitors, 1 ) == 1 )
154         autoArrangeFigures( 3, 4, 1 );
155     elseif ( 1 < size( monitors, 1 ) )
156         autoArrangeFigures( 2, 2, 2 );
157     end
158 end
159
160 fprintf( 1, '\n\n*** Processing Complete ***\n\n' );
161
162
163
164 %% Reference(s)
```
